

Financial Data Mining and Analysis

Course: FICO3002

Semester: Spring 2026

Instructor: Dr. Liqun (Lee) Zhuge

Agenda

- Chapter 2: Introduction to Python modules
 - What are the Python modules/packages?
 - `import math` and `import math as m`, `from math import *`
 - Several ways to install Python modules
 - `pip install numpy`
 - our p4f Python module
 - install a specific module
 - find dependencies of a module
 - `sys.path`, find the location of numpy, add your path
 - `numpy.array([])`
 - `outputHelp2file()`

What are Python modules?

A module could be viewed as a set of programs organized by one specific topic.

A module is also called a package or toolbox.

One simple example

- Assume that we want to calculate the square root of 3

`sqrt(3)`

Traceback (most recent call last):

Cell In[1], line 1

`sqrt(3)`

NameError: name 'sqrt' is not defined

```
In [1]: sqrt(3)
Traceback (most recent call last):

  Cell In[1], line 1
    sqrt(3)

NameError: name 'sqrt' is not defined
```

Using the `sqrt()` function inside the `math` module

```
import math
```

```
math.sqrt(3)
```

```
Out[3]: 1.7320508075688772
```

Several ways to import a module

Method I:

```
import math  
math.sqrt(3)
```

Method II: with a short name

```
import math as m  
m.sqrt(4)
```

Method III: a few functions

```
from math import sqrt  
sqrt(3)
```

Method IV: all functions available

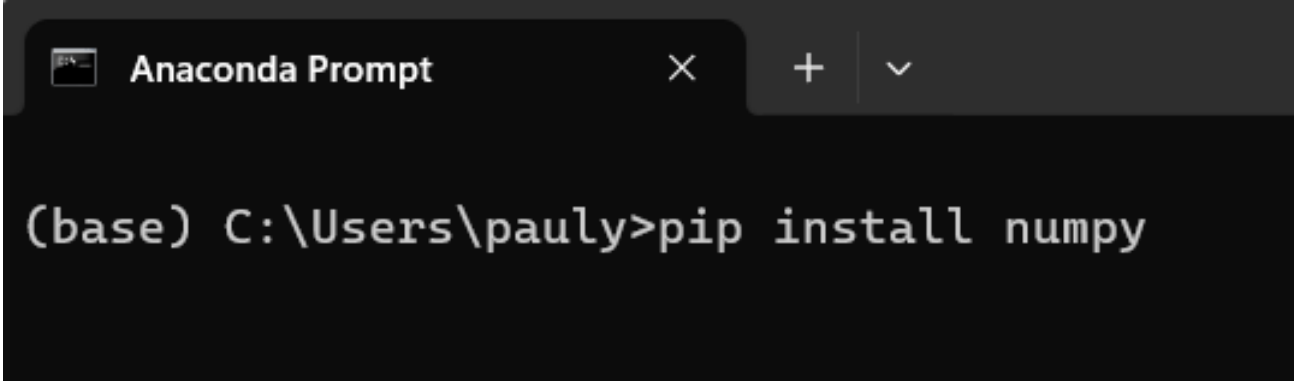
```
from math import *  
sqrt(3)
```

pip install numpy

- Step 1: go to Anaconda prompt

- Step 2: run the following code

`pip install numpy`

- A screenshot of an Anaconda Prompt terminal window. The window has a dark background and a title bar that says "Anaconda Prompt". Inside the terminal, the text "(base) C:\Users\paul>pip install numpy" is displayed, indicating the command being entered in the base environment.

Several ways to install a Python module

For Windows users (assume that we plan to install a Python module called 'pandas')

Click 'Anaconda3'

--> 'Anaconda Prompt (Anaconda3)'

--> [type the following command]

pip install pandas

Windows: Method II

Click 'Anaconda3'

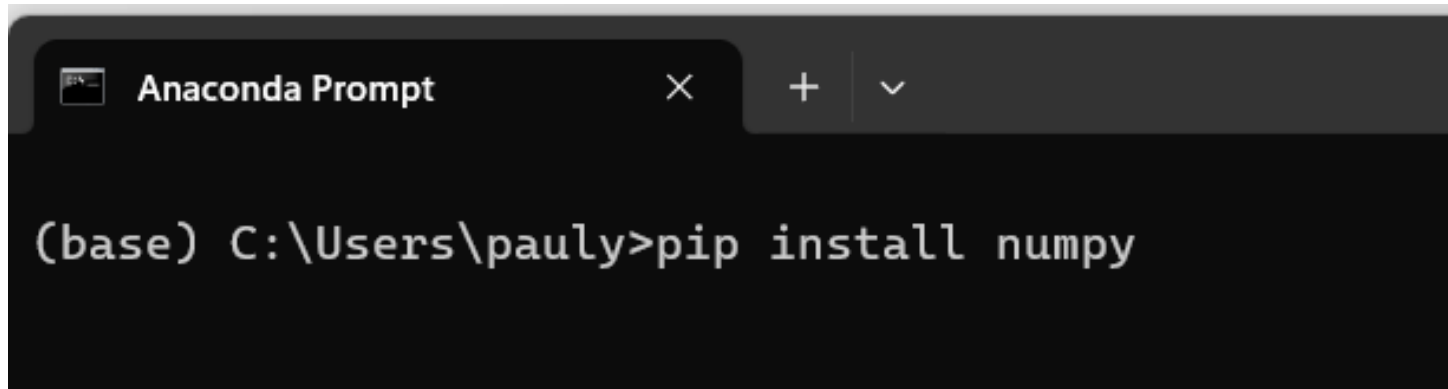
--> 'Anaconda Prompt (Anaconda3)'

--> [type the following command]

conda install yfinance

For Windows users

- Search for Anaconda prompt, shown below.

A screenshot of the Anaconda Prompt application window. The title bar at the top shows 'Anaconda Prompt' with a close button (X) and window control buttons (+ and v). The terminal area has a black background with white text. The prompt shows the user is in the base environment at the C:\Users\paully directory, and they have entered the command 'pip install numpy'.

```
(base) C:\Users\paully>pip install numpy
```

For Mac users, use terminal

Several ways to install a Python module

Windows: Others

```
python -m pip install --upgrade pip
```

```
pip install milk
```

```
pip install 'SomeProject'
```

```
pip install 'SomeProject==1.4'
```

```
pip install 'SomeProject>=1,<2'
```

```
pip install 'SomeProject~=1.4.2'
```

- Windows: to upgrade pip
- ```
python -m pip install --upgrade pip
```

# For Mac Users

For Mac Users

[launch Terminal ]

pip install pandas

.mac(13)

.mac(14)

# pip help

```
(base) C:\Users\paul>pip help
```

## Usage:

```
pip <command> [options]
```

## Commands:

|            |                                                         |
|------------|---------------------------------------------------------|
| install    | Install packages.                                       |
| download   | Download packages.                                      |
| uninstall  | Uninstall packages.                                     |
| freeze     | Output installed packages in requirements format.       |
| inspect    | Inspect the python environment.                         |
| list       | List installed packages.                                |
| show       | Show information about installed packages.              |
| check      | Verify installed packages have compatible dependencies. |
| config     | Manage local and global configuration.                  |
| search     | Search PyPI for packages.                               |
| cache      | Inspect and manage pip's wheel cache.                   |
| index      | Inspect information available from package indexes.     |
| wheel      | Build wheels from your requirements.                    |
| hash       | Compute hashes of package archives.                     |
| completion | A helper command used for command completion.           |
| debug      | Show information useful for debugging.                  |
| help       | Show help for commands.                                 |

# pip help (second part)

```
General Options:
-h, --help Show help.
--debug Let unhandled exceptions propagate outside the main subroutine, instead of logging them
 to stderr.
--isolated Run pip in an isolated mode, ignoring environment variables and user configuration.
--require-virtualenv Allow pip to only run in a virtual environment; exit with an error otherwise.
--python <python> Run pip with the specified Python interpreter.
-v, --verbose Give more output. Option is additive, and can be used up to 3 times.
-V, --version Show version and exit.
-q, --quiet Give less output. Option is additive, and can be used up to 3 times (corresponding to
 WARNING, ERROR, and CRITICAL logging levels).
--log <path> Path to a verbose appending log.
--no-input Disable prompting for input.
--keyring-provider <keyring_provider>
 Enable the credential lookup via the keyring library if user input is allowed. Specify
 which mechanism to use [disabled, import, subprocess]. (default: disabled)
--proxy <proxy> Specify a proxy in the form scheme://[user:passwd@]proxy.server:port.
--retries <retries> Maximum number of retries each connection should attempt (default 5 times).
--timeout <sec> Set the socket timeout (default 15 seconds).
--exists-action <action> Default action when a path already exists: (s)witch, (i)gnore, (w)ipe, (b)ackup,
 (a)bort.
--trusted-host <hostname> Mark this host or host:port pair as trusted, even though it does not have valid or any
 HTTPS.
--cert <path> Path to PEM-encoded CA certificate bundle. If provided, overrides the default. See 'SSL
 Certificate Verification' in pip documentation for more information.
--client-cert <path> Path to SSL client certificate, a single file containing the private key and the
 certificate in PEM format.
--cache-dir <dir> Store the cache data in <dir>.
--no-cache-dir Disable the cache.
--disable-pip-version-check
 Don't periodically check PyPI to determine whether a new version of pip is available for
 download. Implied with --no-index.
--no-color Suppress colored output.
--no-python-version-warning
 Silence deprecation warnings for upcoming unsupported Pythons.
--use-feature <feature> Enable new functionality, that may be backward incompatible.
--use-deprecated <feature> Enable deprecated functionality, that will be removed in the future.
```

# In-class exercise #1: install numpy

- Task #1: install numpy
- Task #2: what is the version of the package
- Task #3: how many functions (objects) in side the package
- Task #4: find the usage of one function and offer one or two examples

# pip upgrade

Step 1: click 'Anaconda Prompt (Anaconda3)

Step 2: issue the following command

[only the top part is shown below]

(base) C:\Users\pyan>pip list

| Package            | Version |
|--------------------|---------|
| -----              | -----   |
| alabaster          | 0.7.12  |
| anaconda-client    | 1.7.2   |
| anaconda-navigator | 2.0.3   |
| anaconda-project   | 0.9.1   |
| anyio              | 2.2.0   |
| appdirs            | 1.4.4   |
| argh               | 0.26.2  |
| argon2-cffi        | 20.1.0  |
| asn1crypto         | 1.4.0   |
| astroid            | 2.5     |

# Finding packages around a topic

- Step 1: go to the following website
  - <https://pypi.python.org/pypi>
- Step 2: type keyword
  - For example for "prediction"

# Python module index: <https://pypi.org>

https://pypi.org



👤 2FA is now required on PyPI

[Read more](#)



[Help](#)

[Sponsors](#)

[Log in](#)

[Register](#)

## Find, install and publish Python packages with the Python Package Index



Or [browse projects](#)

# The keyword is prediction

12 projects for "predition" Did you mean '[prediction](#)'?

Order by

Relevance



**HackDuck 0.1.5**

Machine learning data flow for reproducible data science

Jan 28, 2020



**a2ml 1.0.97**

A powerful API to Automate Machine Learning workflows from multiple vendors.

Jan 20, 2024



**kia-remarketing-supply 1.0.0**

Custom modules for kia\_remarketing\_supply

Oct 5, 2021



**Cad-usd-forecast-model 0.0.1**

Model for forecasting CAD-USD exchange price.

Dec 7, 2022



**nostredame 1.5.0**

forecasts

Jan 26, 2024

# built-in functions

```
print(dir(__builtins__))
```

```
['ArithmeticError', 'AssertionError', 'AttributeError', 'BaseException', 'BaseExceptionGroup',
'BlockingIOError', 'BrokenPipeError', 'BufferError', 'BytesWarning', 'ChildProcessError',
'ConnectionAbortedError', 'ConnectionError', 'ConnectionRefusedError', 'ConnectionResetError',
'DeprecationWarning', 'EOFError', 'Ellipsis', 'EncodingWarning', 'EnvironmentError', 'Exception',
'ExceptionGroup', 'False', 'FileExistsError', 'FileNotFoundError', 'FloatingPointError', 'FutureWarning',
'GeneratorExit', 'IOError', 'ImportError', 'ImportWarning', 'IndentationError', 'IndexError', 'InterruptedError',
'IsADirectoryError', 'KeyError', 'KeyboardInterrupt', 'LookupError', 'MemoryError', 'ModuleNotFoundError',
'NameError', 'None', 'NotADirectoryError', 'NotImplemented', 'NotImplementedError', 'OSError', 'OverflowError',
'PendingDeprecationWarning', 'PermissionError', 'ProcessLookupError', 'RecursionError', 'ReferenceError',
'ResourceWarning', 'RuntimeError', 'RuntimeWarning', 'StopAsyncIteration', 'StopIteration', 'SyntaxError',
'SyntaxWarning', 'SystemError', 'SystemExit', 'TabError', 'TimeoutError', 'True', 'TypeError', 'UnboundLocalError',
'UnicodeDecodeError', 'UnicodeEncodeError', 'UnicodeError', 'UnicodeTranslateError', 'UnicodeWarning',
'UserWarning', 'ValueError', 'Warning', 'WindowsError', 'ZeroDivisionError', '__IPYTHON__', '__build_class__',
'__debug__', '__doc__', '__import__', '__loader__', '__name__', '__package__', '__spec__', 'abs', 'aiter',
'all', 'anext', 'any', 'ascii', 'bin', 'bool', 'breakpoint', 'bytearray', 'bytes', 'callable', 'cell_count',
'chr', 'classmethod', 'compile', 'complex', 'copyright', 'credits', 'debugcell', 'debugfile', 'delattr',
'dict', 'dir', 'display', 'divmod', 'enumerate', 'eval', 'exec', 'execfile', 'filter', 'float', 'format',
'frozenset', 'get_ipython', 'getattr', 'globals', 'hasattr', 'hash', 'help', 'hex', 'id', 'input', 'int',
'isinstance', 'issubclass', 'iter', 'len', 'license', 'list', 'locals', 'map', 'max', 'memoryview', 'min',
'next', 'object', 'oct', 'open', 'ord', 'pow', 'print', 'property', 'range', 'repr', 'reversed', 'round',
'runcell', 'runfile', 'set', 'setattr', 'slice', 'sorted', 'staticmethod', 'str', 'sum', 'super', 'tuple',
'type', 'vars', 'zip']
```

# import pandas as pd

```
print(dir(pd))
```

```
['ArrowDtype', 'BooleanDtype', 'Categorical', 'CategoricalDtype', 'CategoricalIndex',
 'DataFrame', 'DateOffset', 'DatetimeIndex', 'DatetimeTZDtype', 'ExcelFile', 'ExcelWriter',
 'Flags', 'Float32Dtype', 'Float64Dtype', 'Grouper', 'HDFStore', 'Index', 'IndexSlice', 'Int16Dtype',
 'Int32Dtype', 'Int64Dtype', 'Int8Dtype', 'Interval', 'IntervalDtype', 'IntervalIndex', 'MultiIndex',
 'NA', 'NaT', 'NamedAgg', 'Period', 'PeriodDtype', 'PeriodIndex', 'RangeIndex', 'Series', 'SparseDtype',
 'StringDtype', 'Timedelta', 'TimedeltaIndex', 'Timestamp', 'UInt16Dtype', 'UInt32Dtype', 'UInt64Dtype',
 'UInt8Dtype', '__all__', '__builtins__', '__cached__', '__doc__', '__docformat__', '__file__', '__git_version__',
 '__loader__', '__name__', '__package__', '__path__', '__spec__', '__version__', '_config', '_is_numpy_dev',
 '_libs', '_testing', '_typing', '_version', 'annotations', 'api', 'array', 'arrays', 'bdate_range', 'compat',
 'concat', 'core', 'crosstab', 'cut', 'date_range', 'describe_option', 'errors', 'eval', 'factorize', 'from_dummies',
 'get_dummies', 'get_option', 'infer_freq', 'interval_range', 'io', 'isna', 'isnull', 'json_normalize', 'lreshape',
 'melt', 'merge', 'merge_asof', 'merge_ordered', 'notna', 'notnull', 'offsets', 'option_context', 'options', 'pandas',
 'period_range', 'pivot', 'pivot_table', 'plotting', 'qcut', 'read_clipboard', 'read_csv', 'read_excel', 'read_feather',
 'read_fwf', 'read_gbq', 'read_hdf', 'read_html', 'read_json', 'read_orc', 'read_parquet', 'read_pickle', 'read_sas',
 'read_spss', 'read_sql', 'read_sql_query', 'read_sql_table', 'read_stata', 'read_table', 'read_xml', 'reset_option',
 'set_eng_float_format', 'set_option', 'show_versions', 'test', 'testing', 'timedelta_range', 'to_datetime',
 'to_numeric', 'to_pickle', 'to_timedelta', 'tseries', 'unique', 'util', 'value_counts', 'wide_to_long']
```

# help(pd.read\_csv)

[ only the top part is shown below]

Help on function read\_csv in module pandas.io.parsers.readers:

```
read_csv(filepath_or_buffer: 'FilePath | ReadCsvBuffer[bytes] | ReadCsvBuffer[str]' *
sep: 'str | None | lib.NoDefault' = <no_default>, delimiter: 'str | None | lib.NoDefault'
= None, header: "int | Sequence[int] | None | Literal['infer']" = 'infer', names:
'Sequence[Hashable] | None | lib.NoDefault' = <no_default>, index_col: 'IndexLabel |
Literal[False] | None' = None, usecols=None, dtype: 'DtypeArg | None' = None, engine:
'CSVEngine | None' = None, converters=None, true_values=None, false_values=None,
skipinitialspace: 'bool' = False, skiprows=None, skipfooter: 'int' = 0, nrows: 'int |
None' = None, na_values=None, keep_default_na: 'bool' = True, na_filter: 'bool' = True,
verbose: 'bool' = False, skip_blank_lines: 'bool' = True, parse_dates: 'bool |
Sequence[Hashable] | None' = None, infer_datetime_format: 'bool | lib.NoDefault' =
<no_default>, keep_date_col: 'bool' = False, date_parser=<no_default>, date_format: 'str |
None' = None, dayfirst: 'bool' = False, cache_dates: 'bool' = True, iterator: 'bool' =
False, chunksize: 'int | None' = None, compression: 'CompressionOptions' = 'infer',
thousands: 'str | None' = None, decimal: 'str' = '.', lineterminator: 'str | None' = None,
quotechar: 'str' = '"', quoting: 'int' = 0, doublequote: 'bool' = True, escapechar: 'str |
None' = None, comment: 'str | None' = None, encoding: 'str | None' = None,
encoding_errors: 'str | None' = 'strict', dialect: 'str | csv.Dialect | None' = None,
on_bad_lines: 'str' = 'error', delim_whitespace: 'bool' = False, low_memory=True,
memory_map: 'bool' = False, float_precision: "Literal['high', 'legacy'] | None" = None,
storage_options: 'StorageOptions' = None, dtype_backend: 'DtypeBackend | lib.NoDefault' =
<no_default>) -> 'DataFrame | TextFileReader'
```

Read a comma-separated values (csv) file into DataFrame.

# conventional short-cuts

When using a short-cut name, users can use any name.

However, for several of the most frequently used packages, we should use the common short-cut names.

A few examples are given below.

```
import math as m
import numpy as np
import scipy as sp
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
```

# # pip install yfinance

```
import yfinance as yf
```

```
df=yf.download("ibm")
```

```
df.head()
```

```
Out[16]:
```

|            | Open     | High     | Low      | Close    | Adj Close | Volume |
|------------|----------|----------|----------|----------|-----------|--------|
| Date       |          |          |          |          |           |        |
| 1962-01-02 | 7.374124 | 7.374124 | 7.291268 | 7.291268 | 1.541711  | 407940 |
| 1962-01-03 | 7.291268 | 7.355003 | 7.291268 | 7.355003 | 1.555188  | 305955 |
| 1962-01-04 | 7.355003 | 7.355003 | 7.278521 | 7.281708 | 1.539688  | 274575 |
| 1962-01-05 | 7.272148 | 7.272148 | 7.125558 | 7.138305 | 1.509368  | 384405 |
| 1962-01-08 | 7.131931 | 7.131931 | 6.947100 | 7.004461 | 1.481067  | 572685 |

```
df.tail()
```

```
Out[17]:
```

|            | Open       | High       | Low        | Close      | Adj Close  | Volume  |
|------------|------------|------------|------------|------------|------------|---------|
| Date       |            |            |            |            |            |         |
| 2024-02-05 | 185.509995 | 185.779999 | 183.259995 | 183.419998 | 181.762894 | 4379600 |
| 2024-02-06 | 183.550003 | 184.679993 | 183.039993 | 183.410004 | 181.752991 | 3337600 |
| 2024-02-07 | 183.339996 | 184.020004 | 182.630005 | 183.740005 | 182.080002 | 4841200 |
| 2024-02-08 | 182.630005 | 184.550003 | 181.490005 | 184.360001 | 184.360001 | 5161200 |
| 2024-02-09 | 184.440002 | 187.179993 | 183.850006 | 186.339996 | 186.339996 | 5059400 |

# In-class exercise #2: install yfinance

- Task #1: install yfinance
- Task #2: download WalMart historical data set (ticker symbol:WMT)
- Task #3: how many rows, columns
- Task #4: show the top and bottom several observations

# Black-Scholes Call Option model

```
def bsCall(S,X,T,r,sigma):
 from numpy import log, exp, sqrt
 from scipy import stats
 d1=(log(S/X)+(r+sigma*sigma/2.)*T)/(sigma*sqrt(T))
 d2 = d1-sigma*sqrt(T)
 call=S*stats.norm.cdf(d1)-X*exp(-r*T)*stats.norm.cdf(d2)
 return(call)
```

$$C(S, t) = SN(d_1) - Ke^{-rt}N(d_2)$$

Where:

- $d_1: \frac{\ln(S/K) + (r + \frac{\sigma^2}{2})T}{\sigma\sqrt{T}}$
- $d_2: d_1 - \sigma\sqrt{T}$

```
import numpy as np
x=np.array(dir(np))
len(x) # Out[45]: 607
```

```
len(x) # Out[45]: 607
```

```
x[200:250]
```

```
Out[46]:
```

```
array(['datetime_data', 'deg2rad', 'degrees', 'delete', 'deprecate',
 'deprecate_with_doc', 'diag', 'diag_indices', 'diag_indices_from',
 'diagflat', 'diagonal', 'diff', 'digitize', 'disp', 'distutils',
 'divide', 'divmod', 'doc', 'dot', 'double', 'dsplit', 'dstack',
 'dtype', 'e', 'ediff1d', 'einsum', 'einsum_path', 'emath', 'empty',
 'empty_like', 'equal', 'errstate', 'euler_gamma', 'exp', 'exp2',
 'expand_dims', 'expm1', 'expm1x', 'extract', 'eye', 'f2py', 'fabs',
 'fastCopyAndTranspose', 'fft', 'fill_diagonal', 'find_common_type',
 'finfo', 'fix', 'flatiter', 'flatnonzero'], dtype='<U25')
```

# In-class exercise #3: Black-Scholes Model

- Task #1: copy the code
- Task #2: show a few examples
- Task #3: show several ways to input values
- Task #4: add comments or help

```
import numpy as np
help(np.std)
```

Help on function std in module numpy: # many lines are deleted

```
std(a, axis=None, dtype=None, out=None, ddof=0, keepdims=<no value>, *, where=<no value>)
 Compute the standard deviation along the specified axis.
```

Returns the standard deviation, a measure of the spread of a distribution, of the array elements. The standard deviation is computed for the flattened array by default, otherwise over the specified axis.

Examples

-----

```
a = np.array([[1, 2], [3, 4]])
np.std(a)
 1.1180339887498949 # may vary
np.std(a, axis=0)
 array([1., 1.])
np.std(a, axis=1)
 array([0.5, 0.5])
```